

Setor de Educação Profissional e Tecnológica Tecnologia em Análise e Desenvolvimento de Sistemas

DS143 - Estruturas de Dados II

10 - Tabelas de dispersão

Prof. Alex Kutzke

Outubro de 2026

Objetivos da aula

Ao final desta aula você deve ser capaz de:

1. Explicar como a tabela de dispersão transforma a chave em posição, e por que a busca não depende do número de chaves no caso médio;
2. Avaliar uma função de dispersão quanto à distribuição das chaves, e explicar por que m costuma ser primo no método da divisão;
3. Executar à mão a inserção com encadeamento e com sondagem linear, sondagem quadrática e dispersão dupla;
4. Relacionar o fator de carga ao custo médio da busca, e explicar quando e por que a tabela é redimensionada;
5. Explicar por que a remoção em endereçamento aberto exige lápide;
6. Descrever o filtro de Bloom e o tipo de erro que ele admite;
7. Escolher entre tabela de dispersão e árvore de busca a partir das operações que a aplicação precisa.

Este conteúdo cai na Prova 2.

Roteiro

Dispersão

Encadeamento

Endereçamento aberto

Aplicações

Fechamento

Demonstração: o map de Go

```
idade := map[string]int{
    "Ana": 19, "Bruno": 22, "Carla": 20, "Davi": 21,
    "Elisa": 23, "Fábio": 19, "Gabi": 24, "Hugo": 20,
}
for rodada := 1; rodada <= 3; rodada++ {
    for nome := range idade {
        fmt.Printf(" %s", nome)
    }
}
```

```
$ go run mapa.go
1: Fábio Gabi Hugo Ana Bruno Carla Davi Elisa
2: Elisa Fábio Gabi Hugo Ana Bruno Carla Davi
3: Carla Davi Elisa Fábio Gabi Hugo Ana Bruno
```

Nem ordem alfabética, nem ordem de inserção, e cada percurso começa em uma chave diferente.

O tempo de uma busca

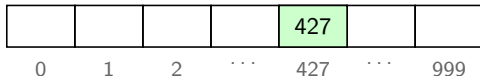
Tempo médio de uma busca bem-sucedida, em nanossegundos:

n		sequencial		binária		map
-----+-----+-----+-----						
1000		429		82		6
10000		1354		67		7
100000		14014		92		13
1000000		150637		158		29

- n mil vezes maior: a sequencial fica 350 vezes mais lenta, a binária, 2 vezes;
- o map é o mais rápido em todas as linhas;
- por que ele também cresce um pouco: no fim da aula.

Endereçamento direto

Chaves de 0 a 999: um vetor de 1000 posições, e a chave 427 fica em $v[427]$.



Busca, inserção e remoção em $O(1)$, sem comparação nenhuma.

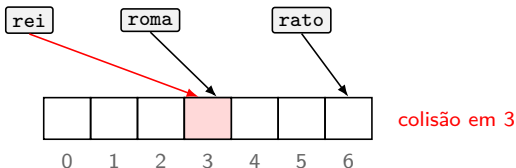
Não serve quando o universo de chaves é grande:

- CPF tem 11 dígitos: 100 bilhões de posições para alguns milhares de clientes;
- chaves de texto nem formam um intervalo de inteiros.

Tabela de dispersão

Uma **função de dispersão** (*hash function*) leva cada chave a uma posição de um vetor de m posições:

$$h : \text{chaves} \rightarrow \{0, 1, \dots, m - 1\}$$



- m é escolhido pelo tamanho do conjunto, e não do universo;
- duas chaves na mesma posição: **colisão**;
- toda tabela de dispersão tem duas partes: a função e uma regra para resolver colisões.

As colisões acontecem cedo

Chaves sorteadas em 365 posições: com **23 chaves**, a chance de alguma colisão já é de 50,7%.

É o paradoxo do aniversário: em uma turma de 23 pessoas, a chance de duas fazerem aniversário no mesmo dia passa de metade.

posições	chaves para 50% de chance
365	23
1000	38
100003	373

A regra de colisão não é um caso raro: ela trabalha o tempo todo.

O que se exige da função de dispersão

Determinística

A mesma chave cai sempre na mesma posição, senão a busca não acha o que a inserção guardou.

Rápida

É calculada em toda operação.

Uniforme

As chaves que a aplicação de fato usa se espalham pelas m posições.

A terceira depende das chaves, e não só da função: `go run funcoes.go`.

Método da divisão: $h(k) = k \bmod m$

1. 1000 chaves múltiplas de 20, com $h(k) = k \% m$

m posições usadas		chaves na posição mais cheia
-----+-----+		-----
100	5	200
97	97	11

- com $m = 100$, o resto de um múltiplo de 20 só pode ser 0, 20, 40, 60 ou 80;
- chaves que avançam de p em p ocupam $m/\text{mdc}(p, m)$ posições;
- com m **primo**, o mdc é 1, e todas as posições são usadas.

Padrões assim são comuns: identificadores de 10 em 10, preços terminados em 0 ou 9, endereços alinhados em 8.

Chaves de texto

palavra		soma		polinomial
-----+-----+-----				
amor		43		52
roma		43		62
mora		43		60
ramo		43		8
omar		43		77

A soma das letras ignora a posição:
todo anagrama colide.

```
func polinomial(s string, m int) int {  
    h := 0  
    for i := 0; i < len(s); i++ {  
        h = (31*h + int(s[i])) % m  
    }  
    return h  
}
```

A palavra como um número na base 31,
pelo método de Horner. É a base do
`String.hashCode()` do Java.

$$h(s) = \left(s_0 \cdot 31^{L-1} + s_1 \cdot 31^{L-2} + \dots + s_{L-1} \right) \bmod m$$

Chaves parecidas entre si

3. 2000 matrículas, de GRR20260001 a GRR20262000, com $m = 997$

função	posições usadas	chaves na posição mais cheia
-----+-----+-----		
soma das letras	28	150
polinomial	634	6
FNV-1a	834	7
sorteio	849	7

- soma: só depende dos quatro dígitos finais, de 1 a 28;
- a polinomial espalha muito melhor, e ainda fica atrás das duas de baixo;
- FNV-1a (pacote hash/fnv) mistura os bits com ou-exclusivo e multiplicação;
- sorteio ignora a chave: é a referência de posições independentes. Mesmo nela, 15% das posições ficam vazias.

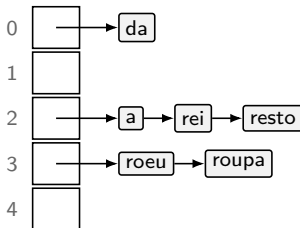
A tabela usada nos exemplos

```
type Tabela struct {  
    posicoes [][]Par  
    n        int // número de pares guardados  
}  
  
func NovaTabela(m int) *Tabela {  
    return &Tabela{posicoes: make([][]Par, m)}  
}  
  
func (t *Tabela) m() int {  
    return len(t.posicoes)  
}  
  
func (t *Tabela) hash(chave string) int {  
    h := 0  
    for i := 0; i < len(chave); i++ {  
        h = (31*h + int(chave[i])) % t.m()  
    }  
    return h  
}
```

m não é um campo: é o comprimento do vetor, e *m()* só o consulta. O *sondagem.go* tem a mesma organização, com o vetor de chaves no lugar do vetor de listas.

Encadeamento

Cada posição guarda a **lista** das chaves que caíram nela.



- inserção: calcula $h(k)$ e acrescenta à lista da posição;
- busca: calcula $h(k)$ e percorre **só** aquela lista;
- remoção: tira da lista.

Busca no encadeamento

```
type Par struct {  
    Chave string  
    Valor int  
}  
  
func (t *Tabela) Busca(chave string) (int, bool) {  
    for _, p := range t.posicoes[t.hash(chave)] {  
        if p.Chave == chave {  
            return p.Valor, true  
        }  
    }  
    return 0, false  
}
```

A lista é um *slice*. Com lista encadeada, como no Cormen, o comportamento é o mesmo.

O programa: contar as palavras de uma frase

```
t := NovaTabela(7)
for _, palavra := range strings.Fields(texto) {
    quantas, _ := t.Busca(palavra)
    t.Insere(palavra, quantas+1)
}
```

- busca a contagem atual (zero, se a palavra é nova) e insere com a contagem mais um. Chave já existente: Insere só troca o valor;
- a tabela começa com 7 posições. A frase tem 14 palavras distintas;
- a oitava, de, deixa 8 chaves em 7 posições, e a tabela cresce:

inserção de "de" $n = 8 > m = 7$, a tabela passa a ter 17 posições

Contando as palavras de uma frase

```
0:
1: [da 1]
2:
3: [roma 1]
4:
5: [de 2]
6: [rato 2]
7:
8: [rainha 1]
9: [o 2]
10: [raiva 1]
11:
12: [a 2] [rei 1] [resto 1]
13: [roeu 2] [roupa 2]
14:
15: [do 2]
16: [e 1]
```

```
go run encadeamento.go
```

*o rato roeu a roupa do rei de roma
e a rainha de raiva roeu o resto da
roupa do rato*

14 chaves, 17 posições.

Seis posições vazias e uma lista de
três chaves, com a tabela menos
que cheia.

Fator de carga e custo

$$\alpha = \frac{n}{m}$$

No encadeamento, α é o comprimento médio das listas, e pode passar de 1.

Com **dispersão uniforme simples** (Cormen):

- busca malsucedida: percorre uma lista inteira, α chaves em média;
- busca bem-sucedida: cerca de metade de uma lista;
- as duas custam $\Theta(1 + \alpha)$: o 1 é calcular $h(k)$.

Consequência

Com α limitado por uma constante, a busca custa $O(1)$ no caso médio. O pior caso continua $\Theta(n)$: todas as chaves na mesma posição.

Redimensionamento

```
func (t *Tabela) redimensiona(m int) {  
    nova := NovaTabela(m)  
    for _, lista := range t.posicoes {  
        for _, p := range lista {  
            i := nova.hash(p.Chave)  
            nova.posicoes[i] = append(nova.posicoes[i], p)  
            nova.n++  
        }  
    }  
    *t = *nova  
}
```

- `encadeamento.go`: cresce quando $\alpha > 1$, para o primeiro primo depois de $2m$;
- h depende de m : toda chave muda de posição e é reinserida;
- um redimensionamento custa $\Theta(n)$.

Custo amortizado

Tabela com 8 posições no início, que dobra quando recebe uma chave a mais do que tem posições. Inserindo 64 chaves:

inserção	posições depois	reinserções
9 ^a	16	8
17 ^a	32	16
33 ^a	64	32
total		56

64 inserções e 56 reinserções: 120 operações, menos de 2 por chave. A soma das reinserções é sempre menor que n , porque cada parcela é o dobro da anterior.

Custo amortizado

Custo médio por operação sobre a sequência inteira. A inserção custa $O(1)$ amortizado, ainda que algumas custem $\Theta(n)$.

O `append` de Go funciona do mesmo jeito: *slice* cheio, aloca um vetor maior e copia os elementos.

Endereçamento aberto

As chaves ficam no próprio vetor, sem listas. Posição ocupada: seguir uma **sequência de sondagem**, que depende da chave:

$$h(k, 0), h(k, 1), h(k, 2), \dots$$

- a inserção para na primeira posição livre;
- a busca segue a mesma sequência, e para ao achar a chave ou uma posição **vazia**;
- a posição vazia prova que a chave não está: se estivesse, teria sido posta ali ou antes;
- uma chave por posição: $\alpha < 1$ sempre.

Três regras de sondagem

Regra	$h(k, i)$	a partir de $h_1(k)$
sondagem linear	$(h_1(k) + i) \bmod m$	$+0, +1, +2, +3, \dots$
sondagem quadrática	$(h_1(k) + i^2) \bmod m$	$+0, +1, +4, +9, \dots$
dispersão dupla	$(h_1(k) + i \cdot h_2(k)) \bmod m$	passo $h_2(k)$

```
func (t *Tabela) posicao(k, i int) int {
    m := t.m()
    h1 := k % m
    switch t.regra {
    case Linear:
        return (h1 + i) % m
    case Quadratica:
        return (h1 + i*i) % m
    default: // Dupla
        h2 := 1 + k%(m-1)
        return (h1 + i*h2) % m
    }
}
```

Dispersão dupla: o passo $h_2(k)$

No sondagem.go:

$$h_2(k) = 1 + (k \bmod (m - 1))$$

$k \bmod (m - 1)$ vai de 0 a $m - 2$; somando 1, o passo fica entre 1 e $m - 1$, e nunca é zero.

chave	h_1	h_2	sequência de sondagem
16	5	$1 + 16 \bmod 10 = 7$	5, 1, 8, 4, ...
27	5	$1 + 27 \bmod 10 = 8$	5, 2, 10, 7, ...

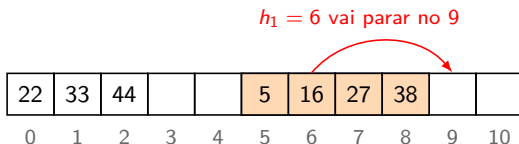
Com $m = 11$: o mesmo h_1 e sequências diferentes. Cada posição é a anterior mais o passo, módulo 11: para o 16, $5 + 7 = 12 \rightarrow 1$, $1 + 7 = 8$, $8 + 7 = 15 \rightarrow 4$.

Sondagem linear

```
$ go run sondagem.go
Sondagem linear, m = 11, h1(k) = k % 11
insere 22: h1 = 0, posições examinadas: 1
insere 33: h1 = 0, posições examinadas: 2
insere 5: h1 = 5, posições examinadas: 1
insere 16: h1 = 5, posições examinadas: 2
insere 27: h1 = 5, posições examinadas: 3
insere 38: h1 = 5, posições examinadas: 4
insere 44: h1 = 0, posições examinadas: 3
    0  1  2  3  4  5  6  7  8  9 10
    22 33 44 . . 5 16 27 38 . .
```

O 5, o 16, o 27 e o 38 têm $h_1 = 5$, e cada um examina uma posição a mais que o anterior.

Agrupamento primário



- as posições 5 a 8 formam um bloco contínuo de posições ocupadas;
- toda chave cujo h_1 caia no bloco vai parar no fim dele, e o aumenta;
- bloco grande recebe mais chaves, e cresce mais depressa que bloco pequeno.

É o **agrupamento primário**, o defeito da sondagem linear.

Quadrática e dupla, mesmas chaves

Sondagem quadrática, $m = 11$, $h_1(k) = k \% 11$

0	1	2	3	4	5	6	7	8	9	10
22	33	.	38	44	5	16	.	.	27	.

Sondagem dupla, $m = 11$, $h_1(k) = k \% 11$

0	1	2	3	4	5	6	7	8	9	10
22	16	27	38	33	5	.	.	.	.	44

- **quadrática:** o bloco não se forma, mas chaves com o mesmo h_1 seguem a mesma sequência (agrupamento secundário). Com m primo, só há garantia de posição livre com $\alpha \leq 1/2$;
- **dupla:** o passo depende da chave. O 16 anda de 7 em 7, o 27 de 8 em 8, o 38 de 9 em 9;
- $h_2(k)$ nunca pode valer 0, e precisa ser primo com m . Com m primo e $1 \leq h_2(k) \leq m - 1$, as duas condições valem.

Custo em função do fator de carga

Busca bem-sucedida

alfa	linear	quadrática	dupla	teoria linear	teoria uniforme
-----+-----+-----+-----+-----+-----					
0.50	1.50	1.43	1.39	1.50	1.39
0.75	2.50	1.96	1.85	2.50	1.85
0.90	5.45	2.79	2.55	5.50	2.56
0.95	10.23	3.49	3.16	10.50	3.15

Busca malsucedida

alfa	linear	quadrática	dupla	teoria linear	teoria uniforme
-----+-----+-----+-----+-----+-----					
0.50	2.49	2.14	2.01	2.50	2.00
0.75	8.62	4.47	4.01	8.50	4.00
0.90	50.46	11.28	9.99	50.50	10.00
0.95	203.56	22.97	19.87	200.50	20.00

Posições examinadas por busca, em média. $m = 100003$, chaves aleatórias. A linear examina mais posições e continua em uso: as posições vizinhas são lidas da memória em bloco, pelo mesmo motivo da página da árvore B, e na quadrática e na dupla cada posição pode exigir uma nova leitura.

As fórmulas

Sondagem linear (Knuth)

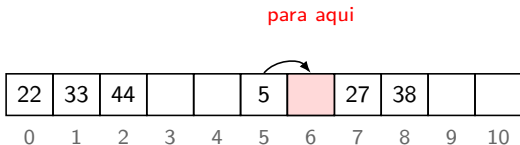
$$\text{bem-sucedida} \approx \frac{1}{2} \left(1 + \frac{1}{1-\alpha} \right) \quad \text{malsucedida} \approx \frac{1}{2} \left(1 + \frac{1}{(1-\alpha)^2} \right)$$

Dispersão uniforme, o caso ideal (Cormen, 11.6 e 11.8)

$$\text{bem-sucedida} \approx \frac{1}{\alpha} \ln \frac{1}{1-\alpha} \quad \text{malsucedida} \approx \frac{1}{1-\alpha}$$

- a dupla fica sobre a curva ideal; a quadrática, um pouco acima;
- o $(1-\alpha)^2$ explica as 200 posições com a tabela 95% cheia;
- crescer em: 1/2 (Sedgewick, linear), 2/3 (dict do Python), 7/8 (map de Go).

Remoção: esvaziar a posição não funciona



O 38 foi inserido na posição 8 depois de passar por 5, 6 e 7. Se a remoção do 16 esvaziar a posição 6, a busca pelo 38 para nela e responde que o 38 **não está** na tabela.

Lápide (*tombstone*)

Um terceiro estado, além de vazia e ocupada: a busca passa por cima, e a inserção pode reaproveitar a posição.

Remoção com lápide

Remoção com lápide, na tabela da sondagem linear:

Remove(16) = true

Busca(38) = true, posições examinadas: 4

0	1	2	3	4	5	6	7	8	9	10
22	33	44	.	.	5	X	27	38	.	.

- as lápides não contam em n , mas alongam as buscas como se fossem chaves;
- muitas remoções acumulam lápides, e a tabela precisa ser reconstruída de tempos em tempos, só com as chaves vivas.

Remoção sem lápide, na sondagem linear

Esvaziar a posição e trazer para ela as chaves seguintes que dependiam dela:

- percorre as posições depois do buraco até achar uma vazia;
- chave com h_1 igual ou anterior ao buraco desce para ele, e o buraco passa para a posição que ela deixou.

Remove(16), sem lápide:

0	1	2	3	4	5	6	7	8	9	10	
22	33	44	.	.	5	16	27	38	.	.	antes
22	33	44	.	.	5	27	38	.	.	.	depois

- o 27 ($h_1 = 5$) desce para a 6, o 38 ($h_1 = 5$) para a 7, e a posição 9 vazia encerra a remoção;
- a busca pelo 38 passa a examinar 3 posições;
- só na sondagem linear. Na quadrática e na dupla, a lápide é a solução usual.

Encadeamento ou endereçamento aberto

	Encadeamento	Endereçamento aberto
Fator de carga	pode passar de 1	sempre menor que 1, com folga
α alto	custo cresce devagar ($1 + \alpha$)	custo dispara perto de 1
Remoção	tira da lista	exige lápide
Memória	ponteiro ou <i>slice</i> por posição, e nós	só o vetor
Acesso à memória	segue ponteiros para outros endereços	posições vizinhas, na linear
Uso	HashMap do Java	dict do Python, map de Go 1.24+

O pior caso provocado

Com a função polinomial de base 31:

$$h(Aa) = 65 \cdot 31 + 97 = 2112 = 66 \cdot 31 + 66 = h(BB)$$

- **bloco**: cada um dos pares Aa e BB. Mesmo valor e mesmo comprimento: trocar um pelo outro não muda o valor da palavra;
- AaAa, AaBB, BBAA e BBBB valem todas 2031744;
- 10 blocos, cada um Aa ou BB: $2^{10} = 1024$ chaves na mesma posição, e a tabela vira uma lista.

Hash flooding (28C3, 2011)

Servidores web em PHP, Java, Python e outras linguagens derrubados por uma requisição com milhares de parâmetros que colidiam todos.

Defesa: uma **semente** sorteada, misturada à função. O map de Go sorteia uma para cada map criado.

Tabela de dispersão e árvore de busca

A função espalha chaves vizinhas por posições distantes: a tabela perde a **ordem**.

Operação	Tabela de dispersão	Árvore balanceada
busca, inserção, remoção	$O(1)$ no caso médio, $\Theta(n)$ no pior	$\Theta(\log n)$ no pior
mínimo e máximo	$\Theta(n)$	$\Theta(\log n)$
percurso em ordem	ordenar antes: $\Theta(n \log n)$	$\Theta(n)$
consulta por faixa	percorre tudo	desce ao início e segue
sucessor	percorre tudo	$\Theta(\log n)$

Frequência de palavras, sessões por identificador, duplicatas: só busca exata. Produtos entre 50 e 100 reais, próximo horário livre: precisa de ordem.

Filtro de Bloom

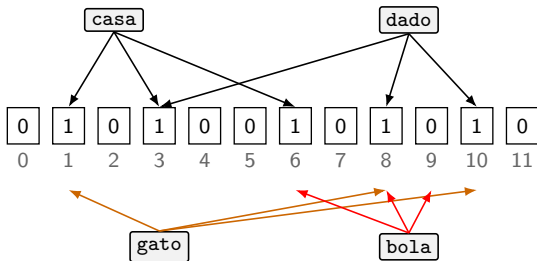
Um vetor de m bits, todos desligados no início, e k funções de dispersão (Bloom, 1970).

- inserir: calcula as k posições e liga os k bits;
- consultar: algum bit desligado, a chave **certamente não está**; todos ligados, **talvez esteja**;
- o filtro não guarda as chaves, só os bits.

Um só tipo de erro

Uma chave nunca inserida pode achar todos os seus bits ligados por outras: **falso positivo**. Os bits de uma chave inserida nunca são desligados: falso negativo, nunca.

Exemplo: $m = 12$ bits, $k = 3$ funções



- inseridas: casa (1, 3, 6) e dado (3, 8, 10). O bit 3 foi ligado pelas duas, e o filtro não registra quem ligou;
- **bola** (6, 8, 9): bit 9 desligado, certamente não está;
- **gato** (1, 8, 10): três bits ligados, por casa e dado. Talvez: falso positivo.

Posições supostas para o exemplo.

Taxa de falsos positivos

$$P \approx \left(1 - e^{-kn/m}\right)^k$$

\$ go run bloom.go
10 bits por chave

k	medida	teoria
1	9.18%	9.52%
2	3.20%	3.29%
3	1.73%	1.74%
4	1.16%	1.18%
5	0.93%	0.94%
6	0.82%	0.84%
7	0.81%	0.82%
8	0.82%	0.85%
9	0.87%	0.91%
10	1.02%	1.02%

- poucas funções: poucos bits conferidos;
- muitas: o vetor enche de bits ligados;
- melhor $k \approx \frac{m}{n} \ln 2 = 6,9$.

Bits por chave: m/n . No exemplo, $12/2 = 6$; aqui, 10, com menos de 1% de erro. Não depende do tamanho da chave, e a taxa de erro depende só dele e de k .

LevelDB e RocksDB podem guardar um filtro por arquivo em disco: a resposta *certamente não está* economiza a leitura.

De volta à demonstração: a ordem do map

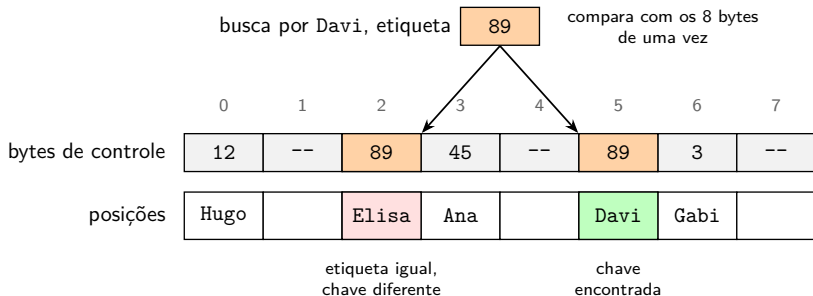
Desde Go 1.24, o map é uma **Swiss Table**: endereçamento aberto.

- uma semente sorteada por map: as posições mudam de uma execução para outra;
- cada percurso sorteia a posição em que começa. As três linhas da demonstração são a mesma sequência circular;
- a especificação diz que a ordem não é especificada, e o sorteio impede que um programa dependa dela.

Para percorrer em ordem, ordenar as chaves:

```
for _, nome := range slices.Sorted(maps.Keys(idade)) {  
    fmt.Println(nome, idade[nome])  
}
```

Swiss Table: um grupo de 8 posições



- grupo: 8 posições (par chave e valor) e 8 bytes de controle, um por posição: vazia (--) ou 7 bits de $h(k)$;
- a parte alta de $h(k)$ escolhe o grupo; os 7 bits da parte baixa são a **etiqueta**;
- busca por Davi, etiqueta 89: uma comparação com os 8 bytes acha as candidatas 2 e 5, e só nelas a chave inteira é comparada;
- Elisa: etiqueta igual por acaso, chance de 1 em 128.

De volta à demonstração: o tempo do map

- chave fora do grupo e grupo com posição vazia: a chave não está;
- grupo cheio: passa a outro grupo, por sondagem quadrática;
- cresce com $7/8$ das posições ocupadas. O fator de carga é alto porque cada passo examina 8 posições de uma vez.

De 6 para 29 ns

O fator de carga fica sempre abaixo de $7/8$, e a busca examina poucas posições em todas as linhas. Com um milhão de chaves, a tabela não cabe na memória *cache*, e cada busca espera pela memória principal.

Escolher entre as estruturas

Estrutura	Busca	Escolha quando
AVL	$\Theta(\log n)$	busca domina, ordem importa
Rubro-negra	$\Theta(\log n)$	uso geral com ordem, muitas alterações
Árvore B / B+	$\Theta(\log_m n)$	a estrutura vive em disco
Trie	$O(\text{tamanho})$	chaves são cadeias, consulta por prefixo
Tabela de dispersão	$O(1)$ médio	só busca exata, sem ordem nem faixa
Filtro de Bloom	$O(k)$	pertinência aproximada, pouca memória

A pergunta que decide entre tabela de dispersão e árvore: a aplicação precisa da ordem entre as chaves?

Para a próxima aula

- exercícios 1 a 12 do roteiro da aula, sem entrega e sem nota. O conteúdo cai na Prova 2;
- rodar `funcoes.go`, `encadeamento.go`, `sondagem.go` e `bloom.go`, alterando as chaves e os tamanhos;
- leitura: Sedgewick, seção 3.4; Cormen, capítulo 11.

Próxima aula: aula prática de tabelas de dispersão em laboratório, com a **Atividade Avaliativa 3**.